

Movimientos bancarios

Guía de uso · unificación, limpieza y clasificación de extractos

Privacidad

Todo el procesamiento ocurre en tu ordenador. La herramienta no abre ninguna conexión de red, no envía datos a ningún servicio y no deja copias fuera de la carpeta del proyecto. Las únicas librerías externas son pandas, openpyxl y xlrd, todas de lectura y escritura local.

Si mueves la carpeta a otro ordenador, cópiala entera y vuelve a instalar las dependencias. No hay nada más que configurar.

Chuleta

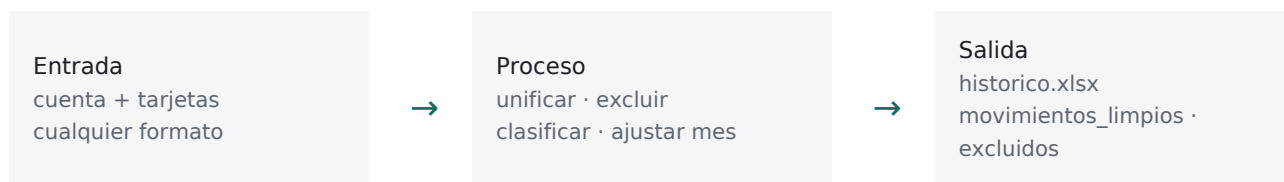
- 1 Descarga del banco los extractos de cuenta y de tarjeta
- 2 Suéltalos en la carpeta entrada/ tal cual: sin renombrar, sin convertir, sin separar por tipo
- 3 Doble clic en ejecutar.bat (Windows), ejecutar.command (Mac) o ejecutar.sh (Linux)
- 4 Revisa el resumen en pantalla: qué ha leído de cada fichero, el resultado, lo que se repite y lo que falta por clasificar; los avisos, juntos al final
- 5 Pulsa 1 para abrir el histórico, 2 para abrir su carpeta o 3 para ejecutar de nuevo tras corregir algo (Intro cierra la ventana)
- 6 Abre tu fichero de contabilidad: los movimientos ya están dentro (si tienes la sincronización activada)

No hay que preparar nada

Los .xls del banco no se convierten: la herramienta detecta el formato real mirando el contenido, no la extensión. Tampoco hay que renombrarlos ni separar cuenta de tarjetas: eso también lo deduce sola.

Qué hace

Lee los movimientos de la cuenta y de todas las tarjetas, los junta en una sola tabla, descarta los movimientos que no quieres contar, los clasifica por categorías y genera un Excel listo para pegar en tu hoja de cálculo. Al terminar, te enseña qué cargos se repiten cada mes o cada año (suscripciones, cuotas, seguros) y cuánto suman al año; sin opinar sobre ninguno. El Excel se abre con Microsoft Excel, OnlyOffice o LibreOffice.



Instalación

Una sola vez: doble clic en `instalar.bat` (en Mac, `instalar.command`; en Linux, `instalar.sh`). Tarda un par de minutos. Prepara las librerías en una carpeta propia dentro de `app/.venv`, sin tocar nada de lo que ya tengas instalado, y a partir de ahí ejecutar `.bat` la usa sola.

Si sale...	Qué hacer
«No encuentro Python»	Instálalo desde python.org/downloads y marca «Add Python to PATH» durante la instalación. Es la casilla que casi todo el mundo se salta y sin ella no funciona nada.
«Falta una librería» al ejecutar	No se ha llegado a instalar. Vuelve a lanzar <code>instalar.bat</code> y mira si termina sin errores.
Mac: no deja abrir el fichero	Clic derecho sobre él → Abrir. Solo la primera vez.
Linux: el doble clic no lo ejecuta	Dale permisos desde una terminal en esa carpeta: <code>chmod +x instalar.sh && ./instalar.sh</code>

Si lo que te han dado es un `eledger.exe`, no hay que instalar nada: doble clic y ya. Y si prefieres hacerlo a mano, `pip install -r requisitos.txt` hace lo mismo. De las tres librerías, `xlrd` solo hace falta para los `.xls` de Excel 97-2003 de verdad; la mayoría de bancos españoles llaman `.xls` a un HTML o a un XML y esos se leen sin ella.

Estructura de carpetas

```

proyecto/
├─ instalar.bat          ← solo la primera vez
├─ ejecutar.bat         ← doble clic para procesar
├─ exportar.bat         ← ZIP para dar a alguien, sin tus datos
│   (en Mac, los tres acaban en .command; en Linux, en .sh)
├─ LEEME.txt
├─ CHANGELOG.md         ← qué cambió en cada versión
├─ GUIA.pdf             ← esto que estás leyendo
├─
├─ entrada/            ← AQUÍ sueltas lo del banco
│   ├─ EXTRACTO_2601.xls
│   ├─ movimientos (3).xls
│   └─ tarjeta_credito.csv
├─
├─ datos/              ← LO INSUSTITUIBLE
│   ├─ historico.xlsx   ← todo lo acumulado
│   └─ copias/          ← copias de seguridad
├─
├─ ajustes/            ← TU configuración
│   ├─ rules.json       ← tus reglas
│   ├─ exclude_patterns.json
│   ├─ categorias.json
│   ├─ mes_contable.json
│   ├─ cuentas.json     ← solo si tienes varias cuentas
│   └─ sincronizar.json
├─
├─ salida/             ← se regenera cada vez
│   ├─ movimientos_limpios.xlsx
│   └─ movimientos_excluidos.xlsx
├─
└─ app/                ← la herramienta. No hay que tocarla.
    ├─ rules_base.json  ← reglas que trae el programa
    └─ VERSION          ← qué versión tienes

```

En el día a día solo tocas entrada/. Los JSON de ajustes/ son los que irás afinando con el tiempo; app/ no se toca salvo que quieras cambiar el comportamiento. datos/ es lo único insustituible: si haces una copia de seguridad de algo, que sea de esa carpeta. Para actualizar a una versión nueva, borra app/ y GUIA.pdf y copia los nuevos encima; las otras cuatro carpetas no se tocan. Si vienes de la versión antigua con todo suelto en una carpeta, la primera ejecución lo recoloca sola y te dice lo que ha movido.

Formatos de entrada

Los bancos llaman «.xls» a cosas muy distintas. La herramienta mira los primeros bytes del fichero y elige el lector adecuado, así que no tienes que hacer nada:

Lo que manda el banco	Qué es en realidad	Se lee
.xls	Excel 97-2003 (BIFF)	sí · requiere xlrd
.xls	una tabla HTML renombrada	sí
.xls	XML de Office (SpreadsheetML 2003)	sí
.xls	un CSV renombrado	sí
.xls	un .xlsx renombrado	sí
.xlsx .csv .ods	lo que dice la extensión	sí

También se resuelven solos: la fila de cabecera (esté donde esté), los nombres de columna según banco (Importe, Importe de la operación, Cantidad...), los acentos, y los importes en formato español (1.234,56 €, (45,00), 12,50-).

Cuenta o tarjeta, y extractos repetidos

No tienes que decirle cuál es cuál. Para cada fichero mira los nombres de las columnas: si hay una de saldo es un extracto de cuenta; si hay una de importe de la operación es de tarjeta. Si el contenido no lo aclara, recurre al nombre del fichero. Por pantalla te dice siempre qué ha decidido y por qué, para que puedas corregirlo si se equivoca.

- `EXTRACTO_2601.xls`: formato=`xls_biff`, cabecera en fila 4, 62 movimientos
→ cuenta (columna «saldo»)
- `tarjeta_credito.csv`: formato=`texto`, cabecera en fila 3, 28 movimientos
→ tarjeta (columna «importe de la operacion»)

Extractos que se solapan

Puedes dejar en entrada/ descargas de meses anteriores sin miedo. Si bajaste abril y luego abril+mayo, los movimientos que aparecen en ambos ficheros se detectan y se cuentan una sola vez.

Repetido no es lo mismo que duplicado

Dos cargos idénticos el mismo día en el mismo sitio son dos gastos reales, no un error. Solo se descartan las repeticiones entre ficheros distintos; las que vienen dentro de un mismo extracto se respetan.

Columnas generadas

Col.	Nombre	Qué contiene
A	<code>fecha</code>	fecha de la operación, sin hora
B	<code>descripcion</code>	el concepto tal cual lo manda el banco
C	<code>importe</code>	número; negativo = gasto
D	<code>tipo</code>	cuenta o tarjeta
E	<code>mes</code>	el mes real de la fecha
F	<code>mes_ajustado</code>	el mes contable (ver «Ajuste de mes»)
G	<code>categoria</code>	resultado de aplicar <code>rules.json</code>
H	<code>categoria_manual</code>	la ÚNICA columna que escribes tú: fuerza una categoría, o «(excluido)» para sacarla de los totales
I	<code>excluido</code>	TRUE si <code>exclude_patterns.json</code> o <code>categoria_manual</code> la sacó de los totales
J	<code>origen</code>	de qué fichero salió la fila
K	<code>regla</code>	qué clave de <code>rules.json</code> la clasificó, o «(manual)»
L	<code>n_rep</code>	0, 1, 2... para distinguir cargos idénticos el mismo día
M	<code>saldo</code>	el saldo que traía el extracto tras esa fila, si lo traía (ver «De dónde parte el Acumulado»)
N	<code>cuenta</code>	el identificador de ajustes/cuentas.json, si tienes más de una cuenta declarada; en blanco si no

No cambies el orden de A-G

Las fórmulas de la hoja MOVIMIENTOS apuntan a columnas concretas: C (importe), F (mes_ajustado) y G (categoria). De H en adelante son de diagnóstico y van al final justamente para no desplazarlas.

rules.json · cómo se clasifica

Es un diccionario de texto a buscar → categoría. Gana la primera regla que casa, así que el orden del fichero importa. La comparación no distingue mayúsculas ni acentos: *nomina* encuentra «NÓMINA».

Hay dos capas de reglas

Fichero	Qué es
ajustes/rules.json	Las tuyas. Mandan sobre la base. No se tocan nunca al actualizar.
app/rules_base.json	La base que viene con el programa: cadenas conocidas en toda España (Mercadona, Endesa, Movistar, Repsol...). Se reemplaza entera con cada versión nueva, así que no la edites: lo que escribas ahí se perderá.

De la base solo entran las claves que tú no hayas escrito ya. Para cambiar una regla de la base, repite esa misma clave en la tuya. Para apagarla, ponla a null. Al ejecutar verás una línea diciendo cuántas reglas hay de cada capa.

Cómo se escribe	Qué significa	Ejemplo
"mercadona"	Inicio de palabra (por defecto). Puede continuar, pero tiene que empezar donde empieza una palabra.	veterin encuentra VETERINARIO
"=dia"	Palabra completa. Para claves cortas y ambiguas.	=dia encuentra DIA pero no MEDIA MARKT
"~dia"	En cualquier sitio. Solo si de verdad quieres pillarlo dentro de otra palabra.	~dia encuentra MEDIA MARKT
"re:..."	Expresión regular, para casos raros.	re:^abono
"_nota"	Se ignora. Sirve para dejarte comentarios dentro del JSON.	—

Qué se puede poner como categoría

Valor	Qué hace
"Comida"	Lo normal: casa y va a esa categoría.
{"+": "Ingresos", "-": "Ocio"}	Según el signo. Un Bizum que recibes es un ingreso; uno que envías, un gasto. Con un solo texto para los dos, los recibidos restaban de Ocio y el mes salía barato sin motivo.
{"-": "Otros"}	Solo un lado. Los movimientos del otro signo ignoran esta regla y siguen buscando más abajo.
null	Apaga la regla. Sirve para desactivar una de la base sin editarla.

Las devoluciones NO necesitan regla de signo

Si te devuelven una compra del Mercadona, que ese abono reste de Comida es exactamente lo correcto. El signo solo hace falta cuando el importe positivo es un concepto distinto del negativo (Bizum recibido / enviado, prestación / cuota), no cuando es la devolución del mismo gasto.

Añadir reglas

```
"decathlon": "Ocio",
"=gym":      "Ocio",
```

Si la categoría es nueva, declárala también en `categorias.json` (como gasto, ingreso o neutra): si no, las reglas que la usan se descartan y se avisa al ejecutar, porque no sumaría en ninguna columna del resumen. Usa = siempre que la palabra sea corta (3-4 letras) o pueda aparecer dentro de otra. Sin ese marcador, bar se comería BARCELONA y vida se comería NAVIDAD.

Probar una regla sin reprocesar todo

Para ver en qué categoría caería un concepto concreto:

```
python app/reglas.py "MEDIA MARKT ONLINE" "BAR LA ESQUINA"

MEDIA MARKT ONLINE -> Otros                [media markt · base]
BAR LA ESQUINA     -> Ocio                  [=bar · base]

python app/reglas.py "BIZUM DE MARTA" 25

BIZUM DE MARTA     25,00 € -> Ingresos      [bizum · base]
```

Si pones un número al final, se usa como importe: es la forma de comprobar las reglas que dependen del signo. Las reglas que salen de la base aparecen marcadas con `· base`. Sin argumentos ejecuta una batería de casos trampa ya preparada.

exclude_patterns.json · qué se descarta

Lista de textos cuyos movimientos no deben contar. Usa la misma sintaxis que `rules.json`. Sirve sobre todo para el cargo mensual de la tarjeta, que si no contarías dos veces: una como recibo en la cuenta y otra como compras sueltas.

```
[
    "pago recibo 77",
    "tarj.crdto 77"
]
```

El 77 del ejemplo son los dígitos con que tu banco identifica la tarjeta en el concepto del recibo. Mira tu extracto y copia los tuyos. El fichero que viene con el programa trae estas instrucciones dentro y ningún patrón activo: es lo primero que hay que configurar.

No hace falta buscarlo a ojo: si el fichero está vacío y hay movimientos de tarjeta, al terminar el script busca en la cuenta un cargo que cuadre con lo que suma la tarjeta ese mes y propone la línea lista para pegar aquí. Con cualquier patrón ya puesto, no dice nada más.

Lo descartado no se pierde: acaba en `movimientos_excluidos.xlsx` para que puedas comprobar que no se ha ido nada de más.

cuentas.json · si tienes más de una cuenta

Solo hace falta si tienes más de una cuenta del mismo tipo (dos cuentas corrientes, por ejemplo). Sin esto, un movimiento idéntico el mismo día en dos cuentas distintas se cuenta una sola vez, como si fuera el mismo.

```
{
  "principal": "principal",
  "ahorro": "ahorro"
}
```

La clave es un trozo del nombre del fichero, no de la descripción de un movimiento: con el ejemplo de arriba, nombra tus extractos de forma que se distingan (principal_042026.xls, ahorro_042026.xls...) y cada uno cae en su cuenta sola. Misma sintaxis que rules.json.

Vacío (como viene de fábrica), todos los ficheros cuentan como si fueran de la misma cuenta: es el comportamiento de siempre, y quien solo tiene una cuenta no nota ningún cambio.

El histórico

historico.xlsx acumula todo lo que has ido procesando, ejecución tras ejecución. Puedes borrar de entrada/ los extractos viejos: lo que ya entró, entrado se queda. Se abre por RESUMEN, y tiene además una hoja _meta con la versión que lo escribió, que no hace falta tocar:

Hoja	Qué contiene
RESUMEN	La matriz mes × categoría con los totales, el balance y el saldo de la cuenta. Debajo, tres gráficos: el Acumulado, los ingresos y gastos de cada mes, y el gasto medio al mes por categoría.
MOVIMIENTOS	Todos los movimientos, uno por fila, incluidos los excluidos (marcados en la columna excluido). La cabecera naranja señala la única columna que puedes escribir tú.

Las categorías no se congelan

El histórico guarda los movimientos en crudo y vuelve a clasificarlos enteros en cada ejecución. Es decir: cuando afines una regla, el cambio se aplica también hacia atrás, a meses de los que ya ni tengas el .xls. Por eso conviene no perder este fichero, y por eso da igual equivocarse con las reglas al principio. La única excepción es lo que escribas en categoria_manual: eso sí se respeta.

Corregir un movimiento suelto

Las reglas leen el texto del concepto, no la fecha, así que no pueden distinguir el BIZUM A MARTA que un mes es la parte del alquiler y otro una cena. Para esos casos está la columna categoria_manual de la hoja MOVIMIENTOS.

- 1 Abre datos/historico.xlsx, hoja MOVIMIENTOS, y busca la línea
- 2 En la columna categoria_manual (cabecera naranja) escribe la categoría que quieras, tal cual aparece en categorias.json
- 3 Guarda, cierra, y vuelve a ejecutar

Esa línea pasa a esa categoría y ahí se queda. La corrección se conserva entre ejecuciones, sobrevive a los extractos nuevos y manda sobre cualquier regla. En la columna regla verás (manual), para que luego sepas por qué esa línea está donde está.

Qué escribes	Qué hace
Una categoría	Esa línea pasa a esa categoría, pase lo que pase con las reglas.
(excluido)	Saca esa línea de los totales, aunque ningún patrón de exclusión la pille.
Nada (vacío)	Comportamiento normal: mandan las reglas.
Algo mal escrito	No se aplica: manda la regla y sale un aviso al ejecutar. Es a propósito: una categoría que no esté en categorias.json no sumaría en ninguna columna del resumen, y el total te descuadraría sin que nada lo dijera.

Escribe solo en esa columna

El resto de la hoja se regenera en cada ejecución y perderías el cambio. Si te equivocas con el nombre de una categoría, se avisa por pantalla. Y por si acaso, antes de cada escritura se guarda una copia del histórico en datos/copias/.

categorías.json · las columnas del resumen

Declara qué categorías espera tu hoja y qué papel juega cada una. Sirve para dos cosas: construir la hoja RESUMEN, y avisarte cuando `rules.json` y el resumen dejan de estar sincronizados.

Bloque	Qué hace
gastos	Una columna cada una. Total Gastos es su suma.
ingresos	Suman en la columna Ingresos.
neutras	Ni gasto ni ingreso: no entran en ningún total. Aquí van los traspasos entre tus propias cuentas, que no son dinero que salga ni entre.
columna_mes	mes o mes_ajustado: con cuál se agrupa el resumen.
etiquetas	Opcional. Nombre interno → texto de columna, para cambiar cómo se ve una columna sin arriesgar el cuadro letra-por-letra con <code>rules.json</code> .
orden_resumen	Opcional. Qué columnas salen y en qué orden, incluidas las de sistema (Balance, Acumulado...), normalmente fijas al final. Sin él, el orden de siempre.
desglosar_ingresos	Opcional. Con <code>true</code> , cada categoría de ingreso tiene su columna, además del total Ingresos. La que se llama «Ingresos» sale como Otros ingresos, y así se la nombra en etiquetas y orden_resumen.

El fallo que esto evita

Si el texto de una categoría en `rules.json` no coincide letra por letra con el del resumen, la columna suma 0 y no protesta nadie. «Higiene» y «Limpieza/Higiene» son distintas; «Fibra/movil» y «Fibra/móvil» también. Al arrancar se comparan los dos ficheros y se avisa de las que sobran, las que faltan y las que solo se diferencian en una tilde o una mayúscula.

Por eso los nombres de categoría van sin tildes: así no hay dos maneras de escribir lo mismo. Y ojo, lo que se declara aquí es el criterio de la fórmula, no el rótulo de la columna: en tu hoja puedes titularla «Gemeliers» mientras la fórmula siga buscando Hijos.

Cómo se calcula el resumen

Total Gastos	= suma de las categorías de gasto
Ingresos	= suma de las categorías de ingreso
Balance	= Ingresos - Total Gastos
Fuera del balance = lo que movió la cuenta sin ser gasto ni ingreso	
Acumulado	= Acumulado del mes anterior + Balance + Fuera del balance

Balance es el resultado del mes por sí solo: lo que ha entrado menos lo que has gastado. Acumulado es el saldo real de tu cuenta al cerrar el mes, el mismo que te da el banco. Fuera del balance explica la diferencia entre los dos: traspasos a otras cuentas tuyas, lo que has excluido (el recibo de la tarjeta, por ejemplo) y las compras con tarjeta que el banco todavía no ha cargado en la cuenta.

Los gastos se muestran en positivo cambiándoles el signo, no con valor absoluto. La diferencia importa cuando una categoría acaba el mes en positivo por una devolución: sale como negativa, en vez de disfrazarse de gasto.

De dónde parte el Acumulado

Si tu extracto de cuenta trae columna de saldo, el Acumulado del primer mes no empieza en 0: empieza en el saldo real que tenía la cuenta antes de tu primer movimiento. Se detecta solo y se avisa por pantalla de qué saldo ha usado. Después comprueba día a día que el saldo calculado coincide con el del extracto: si no, te dice en qué fecha deja de cuadrar y por cuánto (casi siempre falta un extracto entre medias). Sin esa columna empieza en 0, y si solo subes extractos de tarjeta es la suma de los Balances. Con más de una cuenta declarada en cuentas.json, es la suma de todas.

Ajuste de mes · mes_contable.json

Una nómina o una prestación que entra el día 1, 2 o 3 corresponde en realidad al mes anterior: es con lo que has vivido ese mes. Cuando eso pasa, la columna mes_ajustado retrocede un mes y la columna mes conserva el valor real.

fecha	descripcion	importe	mes	mes_ajustado
02/04/2026	NOMINA EMPRESA SL	+2.450,00	2026-04	2026-03
01/05/2026	MUTUA PRESTACION	+1.180,45	2026-05	2026-04
02/05/2026	RECIBO MUTUA	-95,00	2026-05	2026-05
05/04/2026	COMPRA MERCADONA	-62,35	2026-04	2026-04

Fíjate en las dos líneas de la mutua: la prestación que cobras sí se va al mes anterior, pero el recibo que pagas no. Un gasto pertenece al mes en que se paga; solo se mueven los ingresos.

Cómo se configura

```
{
  "dias": 3,                ← 0 lo desactiva del todo
  "palabras": ["nomina", "mutua"],
  "solo_ingresos": true     ← déjalo en true
}
```

Las palabras van en minúsculas y sin tildes. Si dejas solo_ingresos en false, la cuota que le pagas a la mutua se iría al mes anterior junto con la prestación que cobras de ella, y son dos cosas distintas.

Para cuadrar tus meses usa siempre mes_ajustado, no mes.

Sincronizar con tu contabilidad

En vez de copiar y pegar cada mes, la herramienta puede escribir los movimientos directamente en tu fichero. Se configura en sincronizar.json; si ese fichero no existe o dejas archivo vacío, no se sincroniza nada y todo funciona como antes.

```
{
  "archivo": "contabilidad.xlsx",
  "hoja": "MOVIMIENTOS",
  "fila_inicial": 1,
  "columna_inicial": 1,
  "incluir_excluidos": false,
  "copias_de_seguridad": 10
}
```

Qué se conserva de tu libro

Solo se toca la hoja de datos que indiques. Está comprobado que sobreviven las fórmulas, el formato condicional, las celdas combinadas, los colores, los formatos de moneda, los anchos de columna, los paneles inmovilizados, la validación de datos, los gráficos y las imágenes.

Tablas dinámicas y macros sí se perderían

Son lo único que no sobrevive a una reescritura. Por eso, antes de tocar nada, se inspecciona tu libro y se cancela la escritura si encuentra alguna de las dos. Tu fichero se queda como estaba y sigues teniendo `movimientos_limpios.xlsx` para copiar y pegar.

Los seguros

Si pasa esto...	...ocurre esto
Apuntas sin querer a la hoja de totales	Se detecta que tiene fórmulas y no se escribe. La hoja de destino debe ser solo datos.
El nombre de la hoja está mal escrito	Se cancela y se listan las hojas que sí existen.
El libro está abierto en OnlyOffice o Excel	Se avisa de que lo cierres. No se escribe a medias.
El histórico encoge respecto a la vez anterior	Las filas sobrantes se eliminan, no quedan restos debajo.
Cualquier otro fallo	Se hace una copia con fecha en datos/copias/ ANTES de cada escritura. Se guardan las 10 últimas.

Uso en Excel

La hoja RESUMEN ya trae las sumas por mes y categoría. Si prefieres montar las tuyas en tu propio Excel (pegando movimientos_limpios.xlsx en las columnas A-G, o con la sincronización), las columnas están siempre en el mismo sitio, así que tus fórmulas no se rompen de una versión a otra.

```
=SUMAR.SI.CONJUNTO(MOVIMIENTOS!C:C;  
                    MOVIMIENTOS!G:G; "Comida";  
                    MOVIMIENTOS!F:F; "2026-04")
```

C = importe · G = categoría · F = mes_ajustado. Los gastos salen en negativo: ponle un - delante si los quieres en positivo (no uses ABS(), que hace pasar las devoluciones por gasto). Si Excel te da error en los separadores, cambia ; por , según tu configuración regional.

Cuidado con E:E y F:F

Son dos columnas distintas y se parecen mucho: E es el mes real y F es el mes contable. Si el criterio del mes apunta a E, el ajuste de nóminas y prestaciones no se aplica y esos ingresos cuentan en el mes en que entraron. Para que el ajuste sirva de algo, la fórmula tiene que filtrar por F.

Dárselo a otra persona

No comprimas la carpeta

Le estarías dando tu `historico.xlsx`, tus copias y, sobre todo, tu `rules.json`, que es un retrato bastante fino de tu vida: el colegio de los críos, el veterinario, el gimnasio, el casero, la mutua. Y tu `exclude_patterns.json` lleva los dígitos de tu tarjeta.

Usa `exportar.bat` (en Mac, `exportar.command`; en Linux, `exportar.sh`). Genera un ZIP con el programa, los lanzadores, esta guía y unas reglas de partida genéricas. Nada más: no funciona quitando cosas de la carpeta, sino al revés, copiando solo lo que está autorizado, para que un fichero nuevo no se cuele por olvido.

Quien lo reciba, al ejecutarlo por primera vez, se creará su propia configuración a partir de las plantillas. Lo primero que tendrá que hacer es poner en `exclude_patterns.json` el recibo con que su cuenta paga su tarjeta.

Versiones

La versión sale en la primera línea al ejecutar, y queda grabada en la hoja `_meta` de `historico.xlsx` junto con la fecha. Es lo primero que hace falta saber cuando algo no cuadra. `CHANGELOG.md` cuenta qué cambió en cada una.

Si abres un histórico escrito por una versión más nueva que la que tienes, el programa se planta en vez de escribirlo: podría tener columnas que tu versión no conoce y se perderían. Actualiza, o restaura una copia de `datos/copias/`.

Cuando algo no sale

Síntoma	Qué hacer
«Hay ficheros abiertos»	Tienes el histórico abierto en Excel u OnlyOffice. Si has escrito algo, guárdalo, cierra el archivo y pulsa Intro: sigue solo. Con C lo dejas abierto y el resultado va a una copia con fecha al lado, pero el histórico de verdad no se actualiza hasta que lo ejecutes con él cerrado.
«La carpeta entrada/ está vacía»	Los ficheros que empiezan por <code>_</code> , <code>.</code> o <code>~\$</code> se ignoran a propósito, y también los que no tengan una extensión conocida. Comprueba que lo que has soltado no es un <code>.zip</code> ni un <code>.pdf</code> .
Falta un fichero entero	Mira el listado que sale por pantalla: si un fichero no aparece ahí, no se ha leído. Si aparece con un aviso, el aviso dice por qué.
Un extracto se ha tomado por el tipo que no era	Por pantalla se indica el motivo de cada decisión. Si tu banco usa nombres raros, añádelos a <code>PISTAS_CUENTA_COL</code> o <code>PISTAS_TARJETA_COL</code> en <code>bank_io.py</code> . Como apaño rápido, mete la palabra «tarjeta» en el nombre del fichero.
«hace falta la librería xlrd»	Vuelve a lanzar el instalador: la instala en el entorno de la herramienta.
«no encuentro una fila de cabecera»	Tu banco usa nombres de columna que no están mapeados. Ejecuta el diagnóstico (abajo) y añade el nombre a <code>ALIAS_COLUMNS</code> en <code>bank_io.py</code> .
Demasiadas filas en «Otros»	El script agrupa al final los conceptos sin regla por palabra común, de mayor a menor importe, con una línea lista para pegar en <code>rules.json</code> .
Un movimiento cae en la categoría equivocada	Mira la columna K (regla): te dice exactamente qué clave lo clasificó. O bien la afinas con <code>=</code> , o la mueves de sitio en el JSON, porque gana la primera que casa.
Una categoría suma 0 € en el resumen	El texto del criterio de la fórmula tiene que coincidir exactamente con la categoría de <code>rules.json</code> , tilde incluida. «Higiene» y «Limpieza/Higiene» no son lo mismo.

Diagnóstico de un fichero

Enseña qué formato es realmente, dónde está la cabecera y qué columnas ha reconocido. No modifica nada:

```
app\.venv\Scripts\python app\bank_io.py entrada\movimientos.xls (Windows)
app/.venv/bin/python app/bank_io.py entrada/movimientos.xls (Mac, Linux)
```